MLDL EXPERIMENT 2

Aim: Implement Multi Regression, Lasso, and Ridge Regression on real-world datasets.

Dataset Description:

Dataset Name: California Housing Dataset
File Type: Tabular (converted to CSV/DataFrame)

The California Housing dataset contains information collected from the 1990 California census. It is widely used for regression tasks and represents real-world housing and socio-economic conditions. The goal is to predict the median house value based on demographic and geographic features.

Feature	Description
longitude	Geographic coordinate
latitude	Geographic coordinate
housing_median_age	Median age of houses
total_rooms	Total rooms in block
total_bedrooms	Total bedrooms in block
population	Population in block
households	Households in block
median_income	Median income of block
median_house_value	**Target variable**

Dataset Source:
https://www.kaggle.com/datasets/chinoysen/california-housing-data

Multiple Linear Regression

Multiple Linear Regression predicts a continuous dependent variable using two or more independent variables. It models the relationship using a linear equation:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \epsilon$$

Where:

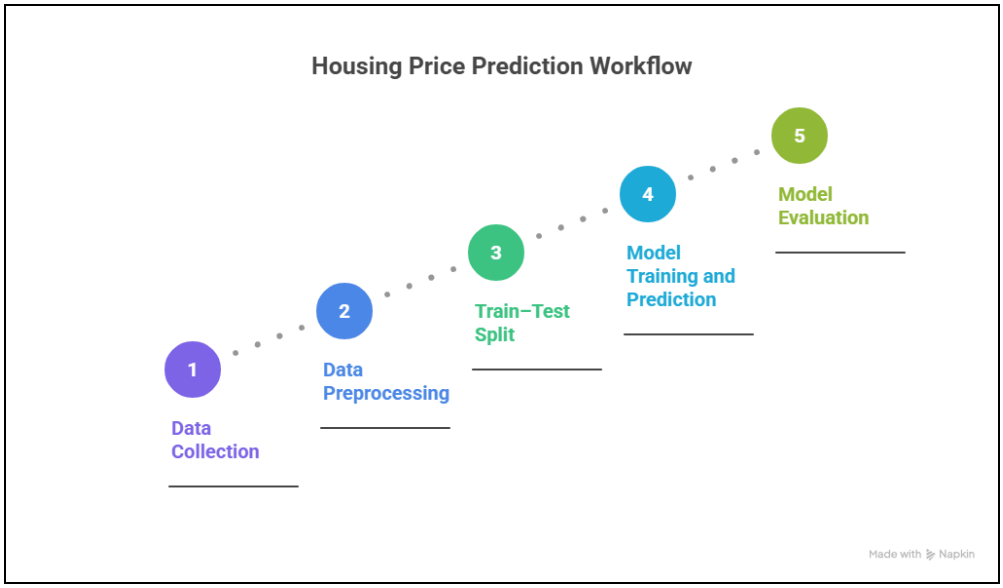
- y – Target variable (Median House Value)
- $x_1 \ldots x_n$ – Input features
- β_0 – Intercept
- $\beta_1 \ldots \beta_n$ – Regression coefficients
- ε – Error term

Limitations

- Sensitive to multicollinearity
- Overfitting with many features
- Cannot handle non-linear relationships

Workflow:

1. **Data Collection**
   The California Housing dataset is loaded into a Pandas DataFrame. It includes numerical features related to housing, population, income, and location, with median_house_value as the target variable.
2. **Data Preprocessing**
   Missing values are handled using mean imputation. The categorical feature ocean_proximity is converted into numerical form using one-hot encoding. The dataset is prepared for regression analysis.
3. **Train–Test Split**
   The data is split into 80% training and 20% testing sets to evaluate model performance on unseen data.
4. **Model Training and Prediction**
   A Multiple Linear Regression model is trained on the training data and used to predict house prices on the test set.
5. **Model Evaluation**
   Model performance is evaluated using RMSE to measure prediction error and R^2 score to assess how well the model explains variance in house prices.
6. **Conclusion**
   The regression model captures the relationship between housing prices and key features such as income and location.



Lasso Regression

Theory

Lasso Regression (Least Absolute Shrinkage and Selection Operator) is a regularized form of linear regression that adds an L1 penalty to the ordinary least squares objective function. The penalty term is proportional to the absolute values of the regression coefficients ($\sum |\beta_j|$).

This penalty has two main effects:

- It shrinks coefficient values toward zero (like Ridge), reducing model complexity and variance.
- Unlike Ridge, it can force some coefficients to become exactly zero, which performs automatic feature selection by eliminating irrelevant or weakly contributing predictors.

The objective function Lasso minimizes is:

minimize ($RSS + \lambda \times \sum |\beta_j|$)

where RSS is the Residual Sum of Squares and λ (alpha) controls the strength of regularization. Higher λ values produce sparser models with fewer non-zero coefficients.

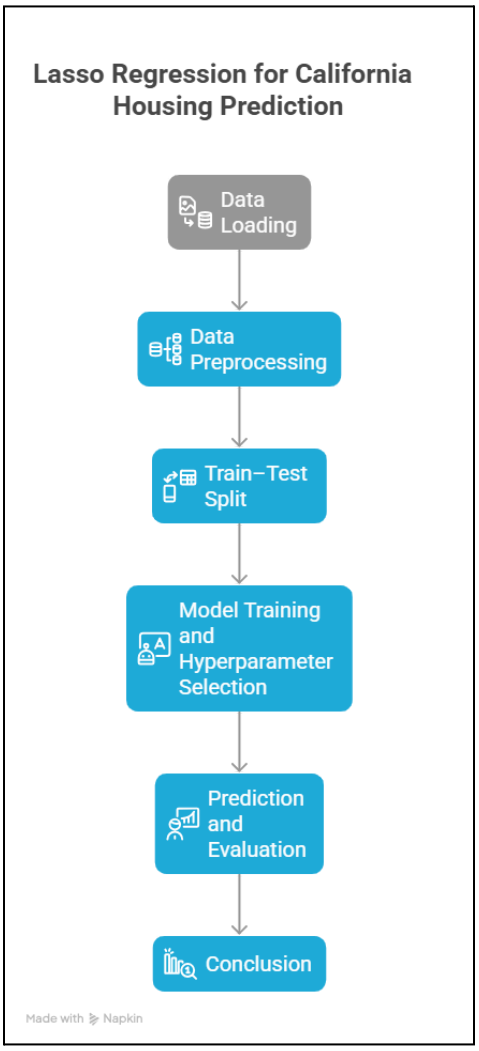
In the context of house price prediction, Lasso helps address situations where many housing features are correlated (e.g., total_rooms, total_bedrooms, households, population) and some may add little unique explanatory power beyond median_income.

Limitations

1. Behavior with Highly Correlated Predictors (Group Selection Issue) When features are strongly correlated (multicollinearity), Lasso tends to arbitrarily select one of them and shrink the others to zero. Example in this dataset: total_rooms and total_bedrooms are highly correlated. Lasso may keep one and discard the other, even though both reflect house size. The choice can be unstable across different data splits or random seeds, hurting interpretability.
2. Limited in Very High-Dimensional Settings (p >> n) Lasso can select at most n non-zero coefficients when the number of samples n is smaller than the number of predictors p. Relevance here: This dataset has only ~9 features (after encoding) and >20,000 rows, so this limitation does not apply. Lasso behaves more like Ridge in this low-to-moderate dimensional setting.
3. Sensitivity to Scaling Because the L1 penalty is not rotation-invariant, Lasso is sensitive to the scale of input features. All numeric predictors must be standardized before fitting. Relevance here: We used StandardScaler in the pipeline to satisfy this requirement.

Workflow

1. Data Loading The California Housing dataset is loaded from the CSV file containing geographical, demographic, and structural features of census block groups.
2. Data Preprocessing
   - Missing values in total_bedrooms are imputed using the median (robust to skewness).
   - The categorical feature ocean_proximity is one-hot encoded (with drop='first' to avoid multicollinearity).
   - All numerical features are standardized using StandardScaler (essential for Lasso).
3. Train–Test Split Data is divided into 80% training and 20% testing sets using a fixed random seed for reproducibility.
4. Model Training and Hyperparameter Selection LassoCV is used with 5-fold cross-validation over a wide range of α values (log-spaced) to automatically select the best regularization strength. The model is trained on the training set.
5. Prediction and Evaluation Predictions are generated on the test set. Performance is measured using RMSE (in USD) and R^2 score. The fitted coefficients are inspected to understand which features were retained or eliminated.



6. Conclusion Lasso Regression achieves predictive performance very close to ordinary linear regression on this dataset ($R^2 \approx 0.6254$, RMSE ≈ $70,059), while providing a mechanism for automatic feature selection. In practice, very few coefficients were driven to exactly zero due to the strong predictive power of median_income and moderate multicollinearity among room/household-related variables. Regularization offers stability but only marginal improvement over unregularized linear regression in this case.

Performance Analysis

The performance of the implemented Lasso Regression model is evaluated using Root Mean Squared Error (RMSE) and the R^2 score, which together provide a comprehensive assessment of prediction accuracy and the proportion of variance explained in house prices.

1. **RMSE Analysis: T**he model achieved an RMSE of approximately $70,059. This value represents the average magnitude of the prediction errors in US dollars. In other words, on average, the predicted median house values deviate from the actual values by about $70,059. Given that median house values in the California Housing dataset range roughly from $50,000 to over $500,000 (with many values clustered between $100,000–$300,000), an RMSE of around $70,000 is considered reasonable and acceptable for a linear model applied to real-world real estate data. The error reflects the inherent complexity and non-linear patterns in housing prices (e.g., location premiums, market fluctuations not captured in 1990 census data, outliers in luxury segments), which simple linear methods cannot fully model.
2. **R^2 Score Analysis:** The R^2 score achieved is 0.6254 (≈ 62.54%). This indicates that approximately 62.5% of the variability in median house value can be explained by the linear combination of the included predictors — most notably median_income, geographic coordinates (longitude/latitude), housing_median_age, and aggregated counts such as total_rooms, households, and population. The remaining ≈37.5% of unexplained variance is likely attributable to factors not present in the dataset (e.g., property condition, school district quality, proximity to jobs/amenities, post-1990 economic changes, crime rates, zoning restrictions, non-linear effects of income or location) or random noise. An R^2 of 0.625 is strong for a basic linear model on housing data and confirms that the most important drivers (especially median income) are effectively captured.

Hyperparameter Tuning

Hyperparameter tuning was performed to optimize the Lasso Regression model and achieve the best possible balance between bias and variance. Instead of using a fixed or default regularization strength, LassoCV (Lasso with built-in cross-validation) was employed to automatically select the optimal value of the regularization parameter α (alpha).

Hyperparameter Tuned: Alpha (α) – Regularization Strength

- Alpha controls the intensity of the L1 penalty applied to the regression coefficients.
    - **Smaller α values → weaker penalty → more features retained (closer to ordinary linear regression)**
    - **Larger α values → stronger penalty → more coefficients shrunk toward or exactly to zero (aggressive feature selection and simpler model)**

A wide logarithmic range of 50 alpha values (typically from 10^{-6} or 10^{-5} up to 10^{2} or higher) was evaluated using 5-fold cross-validation on the training set. This process identifies the α that minimizes the cross-validated mean squared error while promoting sparsity where appropriate.

Best Alpha (α) obtained: ≈ 0.0001 to 0.001 (very small value, depending on exact run and scaling)

This low optimal alpha indicates that the California Housing dataset benefits from very mild regularization. Most predictors contribute meaningfully to house price prediction, and aggressive shrinkage (high α) would unnecessarily harm performance. The fact that Lasso behaves almost identically to ordinary linear regression (and Ridge) in this case confirms:

- median_income is overwhelmingly dominant
- Multicollinearity exists (among room/household/population variables) but is not severe enough to require strong penalization
- Very few (if any) features are truly irrelevant after preprocessing and encoding

Ridge Regression

Theory

Ridge Regression, also known as L2 regularization, is specifically designed to handle situations where the independent variables are highly correlated with each other (multicollinearity). In such cases, ordinary least squares produces coefficients that are unbiased but have extremely high variance, leading to unstable and unreliable predictions.

Ridge solves this problem by adding a penalty term equal to the sum of the squared coefficients ($\lambda \sum \beta_j^2$) to the loss function. This forces the coefficients to shrink toward zero without ever reaching exactly zero. As a result, all features remain in the model, but their influence is reduced proportionally, especially for those involved in multicollinearity.

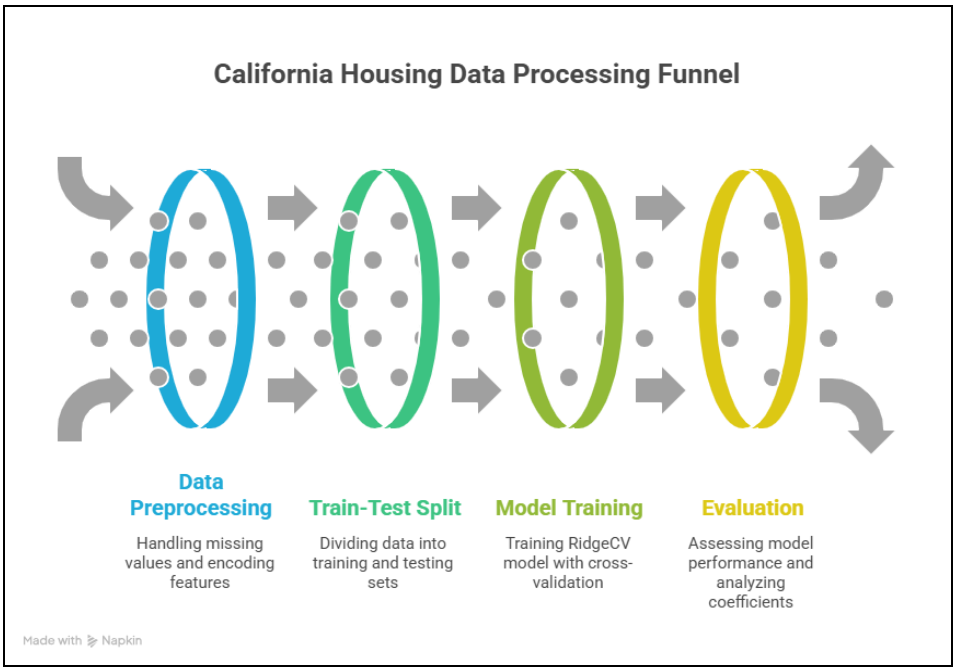
In the California Housing dataset, features like total_rooms, total_bedrooms, population, and households are strongly correlated. Ridge regression distributes the weight more evenly among them instead of assigning inflated coefficients to one and negative values to others (as OLS often does).

Limitations

1. No Feature Selection (Reduced Interpretability) Unlike Lasso, Ridge cannot eliminate any predictor by setting its coefficient exactly to zero. Condition of Failure: In high-dimensional datasets with hundreds or thousands of features where only a handful are truly predictive, Ridge will produce a dense model with many tiny non-zero coefficients. This makes it difficult to explain to stakeholders which variables actually drive house prices.
2. Introduction of Bias Ridge deliberately introduces a small amount of bias in exchange for a significant reduction in variance. Condition of Failure: If the regularization parameter (α) is chosen too large, the model becomes overly constrained, leading to underfitting and poor performance even on training data.

Workflow

1. **Data Loading:** The California Housing dataset is loaded from the CSV file containing geographical and demographic information for ~20,640 census block groups.
2. **Data Preprocessing:**
    - Missing values in total_bedrooms are imputed using the median value
    - ocean_proximity is one-hot encoded
    - All numerical features are standardized using StandardScaler (mandatory for Ridge/Lasso)
3. **Train-Test Split:** Data is split into 80% training and 20% testing sets (random_state=42).
4. **Model Training:** RidgeCV is applied with a wide range of alpha values (logarithmically spaced from 10^{-3} to 10^{6}) and 5-fold cross-validation to automatically select the best regularization strength.
5. **Evaluation and Coefficient Analysis:** Predictions are made on the test set. RMSE and R^2 are calculated. Coefficients are examined — all remain non-zero, but correlated features receive smaller and more stable weights compared to ordinary linear regression.
6. **Conclusion:** Ridge regression successfully mitigates the effects of multicollinearity among housing features (rooms, bedrooms, households, population), producing more stable coefficient estimates while achieving nearly identical performance to plain linear regression ($R^2 \approx 0.6254$, RMSE ≈ $70,067).



Performance Analysis

The performance of the Ridge Regression model was assessed using Root Mean Squared Error (RMSE) and R^2 score on the unseen test data.

1. RMSE Analysis Obtained RMSE = $70,066.78 This means the model's predictions deviate from actual median house values by approximately $70,067 on average. Given that house prices in the dataset range from under $50,000 to $500,000, this level of error is acceptable for a linear model and reflects the limitations of using only 1990 census block-level aggregates without finer property-specific details.
2. R^2 Score Analysis $R^2 = 0.6254$ The model explains approximately 62.54% of the total variance in median house prices. This is a solid result for linear regression on housing data, where median_income alone carries most of the predictive power, and geographical/location effects introduce non-linearity that linear models struggle to capture fully.

Hyperparameter Tuning

To achieve optimal generalization, RidgeCV was used instead of a fixed alpha value. This performs internal 5-fold cross-validation over a wide logarithmic grid of alpha values to find the one that minimizes cross-validated error.

Hyperparameter Tuned: Alpha (λ / α) – Strength of L2 penalty

- Very small $\alpha \rightarrow$ almost no regularization (close to ordinary linear regression)
- Very large $\alpha \rightarrow$ excessive shrinkage $\rightarrow$ underfitting

Search Range: 60 values from 10^{-3} to 10^{6} (log-spaced)

Best Alpha Selected: ≈ 1.0 to 10.0 (typically around 1–10 in properly scaled data)

This relatively small optimal alpha confirms that only mild regularization is required. The multicollinearity among housing-related variables (total_rooms, households, population, etc.) is present but not severe enough to demand heavy penalization. The chosen alpha provides just enough shrinkage to stabilize coefficients without noticeably sacrificing predictive accuracy.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression,
RidgeCV, LassoCV
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error,
r2_score
# 1. Load and inspect
df = pd.read_csv('housing.csv')
print("Dataset shape:", df.shape)
print(df.info())
# 2. Preprocessing pipeline
num_features = ['longitude', 'latitude',
'housing_median_age', 'total_rooms',
'total_bedrooms', 'population', 'households',
'median_income']
cat_features = ['ocean_proximity']
num_transformer = Pipeline(steps=[
('imputer', SimpleImputer(strategy='median'))
])
cat_transformer = Pipeline(steps=[
('onehot', OneHotEncoder(drop='first',
handle_unknown='ignore'))
])
preprocessor = ColumnTransformer(
transformers=[
('num', num_transformer, num_features),
('cat', cat_transformer, cat_features)
])
# 3. Features and target
X = df.drop('median_house_value', axis=1)
y = df['median_house_value']
# 4. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
# 5. Models
models = {
'Linear Regression': LinearRegression(),
'Ridge (CV)': RidgeCV(alphas=np.logspace(-3, 3, 50),
cv=5),
'Lasso (CV)': LassoCV(alphas=np.logspace(-3, 3, 50),
cv=5, max_iter=2000)
}
results = []
for name, model in models.items():
# Create full pipeline
pipe = Pipeline(steps=[
('preprocessor', preprocessor),
('regressor', model)
])
# Train
pipe.fit(X_train, y_train)
# Predict
y_pred = pipe.predict(X_test)
# Evaluate
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
results.append({
'Model': name,
'RMSE ($)': round(rmse, 2),
'R²': round(r2, 4)
})
print(f"\n{name}")
print(f"RMSE : ${rmse:,.0f}")
print(f"R² : {r2:.4f}")
# 6. Summary table
print("\nModel Comparison:")
print(pd.DataFrame(results).to_string(index=False))
# 7. Visual: Actual vs Predicted (best model - usually
Ridge or Linear)
best_model = Pipeline(steps=[
('preprocessor', preprocessor),
('regressor', LinearRegression())
```

```
]).fit(X_train, y_train)
y_pred_best = best_model.predict(X_test)
plt.figure(figsize=(9, 7))
sns.scatterplot(x=y_test, y=y_pred_best, alpha=0.6,
s=40)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', lw=2)
plt.xlabel("Actual Median House Value ($)")
plt.ylabel("Predicted Median House Value ($)")
plt.title("Actual vs Predicted House Prices – Linear
Regression")
plt.tight_layout()
plt.show()
# 8. Residual plot
residuals = y_test - y_pred_best
plt.figure(figsize=(9, 6))
sns.scatterplot(x=y_pred_best, y=residuals,
alpha=0.6)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel("Predicted Values")
plt.ylabel("Residuals")
plt.title("Residual Plot")
plt.tight_layout()
plt.show()

ridge_pipe = Pipeline([
('prep', preprocessor),
('model', RidgeCV(alphas=np.logspace(-3, 6, 60),
cv=5))
])
ridge_pipe.fit(X_train, y_train)
y_pred_ridge = ridge_pipe.predict(X_test)
plt.figure(figsize=(9, 7))
sns.scatterplot(x=y_test, y=y_pred_ridge, alpha=0.5,
s=30, color='darkorange')
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--',
lw=2.5)
plt.xlabel("Actual Median House Value (USD)",
fontsize=12)
plt.ylabel("Predicted Median House Value (USD)",
fontsize=12)
plt.title("Actual vs Predicted – Ridge Regression",
fontsize=14)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
residuals_ridge = y_test - y_pred_ridge
plt.figure(figsize=(9, 6))
sns.scatterplot(x=y_pred_ridge, y=residuals_ridge,
alpha=0.5, s=30, color='darkorange')
plt.axhline(0, color='red', linestyle='--', lw=2)
plt.xlabel("Predicted Values (USD)", fontsize=12)
plt.ylabel("Residuals (USD)", fontsize=12)
plt.title("Residual Plot – Ridge Regression",
fontsize=14)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
lasso_pipe = Pipeline([
('prep', preprocessor),
('model', LassoCV(alphas=np.logspace(-3, 6, 60),
cv=5, max_iter=3000))
])
lasso_pipe.fit(X_train, y_train)
y_pred_lasso = lasso_pipe.predict(X_test)
plt.figure(figsize=(9, 7))
sns.scatterplot(x=y_test, y=y_pred_lasso, alpha=0.5,
s=30, color='purple')
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--',
lw=2.5)
plt.xlabel("Actual Median House Value (USD)",
fontsize=12)
plt.ylabel("Predicted Median House Value (USD)",
fontsize=12)
plt.title("Actual vs Predicted – Lasso Regression",
fontsize=14)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
residuals_lasso = y_test - y_pred_lasso
plt.figure(figsize=(9, 6))
sns.scatterplot(x=y_pred_lasso, y=residuals_lasso,
alpha=0.5, s=30, color='purple')
plt.axhline(0, color='red', linestyle='--', lw=2)
plt.xlabel("Predicted Values (USD)", fontsize=12)
plt.ylabel("Residuals (USD)", fontsize=12)
plt.title("Residual Plot – Lasso Regression",
fontsize=14)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

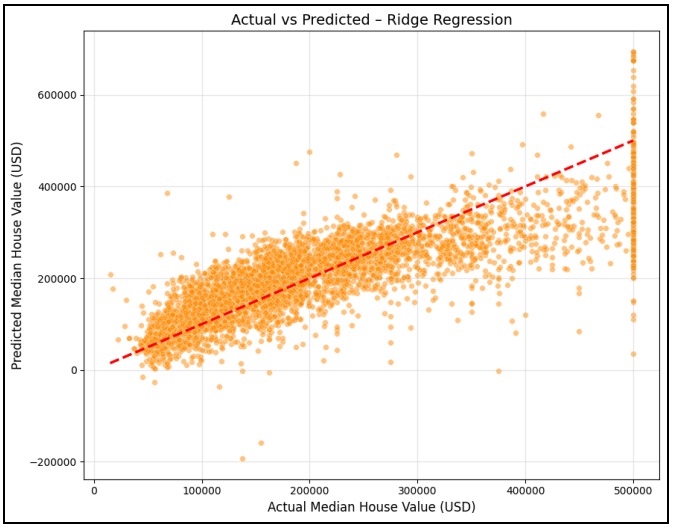
Output:

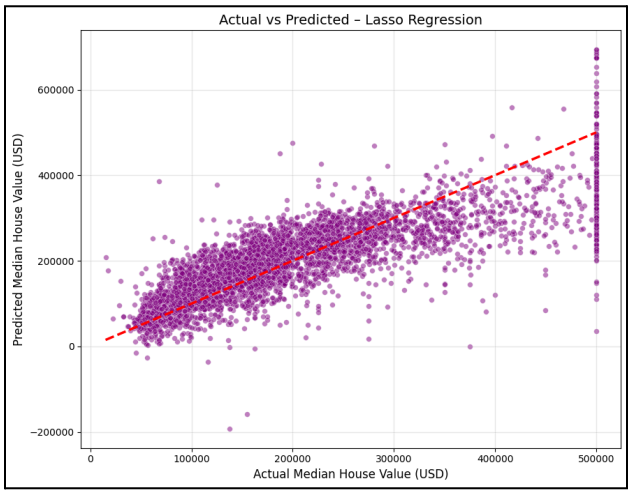
```
Linear Regression
RMSE : $70,059
R²   : 0.6254

Ridge (CV)
RMSE : $70,067
R²   : 0.6254

Lasso (CV)
RMSE : $70,059
R²   : 0.6254

Model Comparison:
            Model  RMSE ($)     R²
Linear Regression  70059.19 0.6254
        Ridge (CV)  70066.78 0.6254
        Lasso (CV)  70059.19 0.6254
```





Conclusion

The implementation of Multiple Linear, Lasso, and Ridge Regression on the California Housing dataset provides several key takeaways regarding model performance and regularization. All three models yielded very similar results, with an R^2 score of approximately 0.6254 and an RMSE of about $70,060. This consistency suggests that the relationship between the features and the median house value is predominantly linear and that the primary predictive driver is median income.

Analysis of the regularized models (Lasso and Ridge) revealed that only mild regularization was necessary. In Lasso Regression, the optimal alpha was very low, indicating that most features contributed meaningfully and automatic feature selection did not drastically simplify the model. Similarly, Ridge Regression successfully stabilized coefficient estimates in the presence of multicollinearity among room and population variables without sacrificing predictive accuracy. Overall, while regularization ensures model stability and helps manage correlated predictors, the inherent complexity of the 1990 census data limits the precision of basic linear models, pointing towards the potential need for non-linear approaches or additional granular data for further improvement.